# Advanced Database Systems Portfolio

Documentation and Technical Reflection: CarePoint Specialist Clinic

| | |
|---|---|
| **Student** | FARAYOLA OLUWADAMILOLA |
| **Student ID** | 301900517 |
| **Course** | MIT 8103 Advanced Database Systems, Miva Open University, 2026/2027 First Semester Continuous Assessment |
| **Case study** | CarePoint Specialist Clinic (simulated), Lagos: Ikeja, Lekki and Yaba branches |
| **GitHub repository** | https://github.com/ofarayola/advanced-database-portfolio |
| **Tools** | PostgreSQL 18, MongoDB 9.0.2 (mongosh 2.12), Python 3.13 (Faker, psycopg2, pymongo), Git/GitHub, VS Code, dbdiagram.io |

## 1. Selected organisation

For this portfolio, I worked with **CarePoint Specialist Clinic**, a clinic I made up but based on how outpatient clinics in Lagos actually run. It has three branches (Ikeja, Lekki and Yaba) sharing one database.

- **Users:** front-desk staff, doctors, nurses, lab scientists, pharmacists, cashiers and management.
- **Data:** patients and their HMO cover, doctors and departments, appointments, consultations and vital signs, lab tests, prescriptions, drug stock at each branch, invoices and payments.
- **Main operations:** booking, consulting, ordering tests, dispensing, billing and reporting. I picked a clinic on purpose. Two pharmacists dispensing the same drug, lab results that look different for every test, and three branches that all need the system up at once gave me a real problem to solve in every portfolio, not just the first one.

## 2. Summary of the five portfolio activities

| Portfolio | What I did | Key results and evidence |
|---|---|---|
| **P1 Design and Modelling** | Modelled the clinic in 18 tables and 1 view (patients, HMO history, appointments, consultations, vital signs, lab orders, prescriptions, drug stock per branch, invoices, payments). Normalised a visit record from UNF to 3NF/BCNF. Business rules enforced as named CHECK, UNIQUE and FOREIGN KEY constraints, plus a partial unique index for “one active HMO per patient”. | ER diagram; schema.sql; 16 negative and 3 positive constraint tests, all passed. Two redundancies (invoice.status, invoice.patient_id) kept as documented trade-offs and controlled later in P3. |
| **P2 Query Optimisation** | Generated realistic data with Python (20,000 patients, 60,000 appointments, ~41,600 invoices; seed 42). Wrote 8 role-based queries (joins, aggregation, window functions) and compared EXPLAIN (ANALYZE, BUFFERS) before and after indexing, median of 3 runs. | Q1 17× and Q2 20× faster; foreign keys are not indexed automatically; rewriting DATE(start_time) as a range let the index be used; partial indexes; PostgreSQL 18 skip scan observed. |
| **P3 Transactions and Concurrency** | Nine two-session demos: COMMIT, ROLLBACK, SAVEPOINT; lost update and three fixes (atomic update, SELECT … FOR UPDATE, REPEATABLE READ); non-repeatable read; deadlock and lock ordering; write skew (double payment); triggers. | Lost update silently lost 10 tablets; identical steps under REPEATABLE READ were refused. SERIALIZABLE blocked a ₦21,500 double payment. Triggers kept invoice.status correct and blocked overpayment even at READ COMMITTED. |
| **P4 NoSQL (MongoDB)** | Redesigned the 18 tables as 6 document collections (embed what is read together; reference what grows without limit; polymorphic lab results). Migrated all data with a verified Python script; re-ran the P2 questions; tested indexes, atomic updates and $jsonSchema validation. | All counts matched PostgreSQL; the same 8 queries gave identical answers; Q1 110 ms → 1 ms with an index. Read-then-write lost 4,260 of 10,000 updates; $inc lost 0. Validation rejected bad data but cannot check references (no foreign keys). |

| Portfolio | What I did | Key results and evidence |
|---|---|---|
| **P5 Distributed and Cloud** | Built a 3-node MongoDB replica set (rs0) from config files. Tested replication, failover, write concerns and a multi-document transaction; wrote a sharding and cloud-deployment design. | Data written once appeared on all 3 nodes. Clean primary shutdown: 0 of 60 writes lost; crash: 13.0 s without writes, then automatic recovery. With 1 of 3 nodes left, MongoDB refused writes (CP). Transaction: 4054 → 4044 atomically; failed dispense fully rolled back. |

## 3. What I took away from the five portfolios

- **The worst errors were the silent ones.** My lost-update demo in P3 quietly lost 10 tablets of Artemether, and in P4 the same mistake lost 4,260 out of 10,000 updates. Nothing on the screen said anything was wrong. I only noticed because I kept a running stock count on paper. Every fix that worked put the check inside the database rather than trusting the application.
- **The same problem kept coming back in different clothes.** In P1 I chose to store invoice.status and noted it could drift. In P3 it did drift, and I fixed it with a trigger. In P4 my MongoDB copy and my PostgreSQL copy ended up disagreeing (4054 against 4074). P5 was where I finally saw a database keep several copies in agreement by itself.
- **Neither database "won".** PostgreSQL felt safer for money and stock: it shouts when something is wrong and has foreign keys. MongoDB was much nicer for reading a whole visit at once and for lab results. If CarePoint were real, I would keep PostgreSQL as the main system and use MongoDB for lab results and the doctor's view.
- **Availability is something you can measure.** Stopping the primary politely lost nothing. Killing it lost 5 writes and gave 13 seconds where no branch could save anything. When only one node was left, MongoDB refused writes altogether rather than risk two primaries.

## 4. Selected evidence

Everything (scripts, config files, query plans, outputs and screenshots) is in the GitHub repository, and each portfolio folder has its own README explaining what I did and how I tested it. Here are four screenshots from my own runs:



*Figure 1. P1 ER diagram (18 tables, grouped in bands by area)*

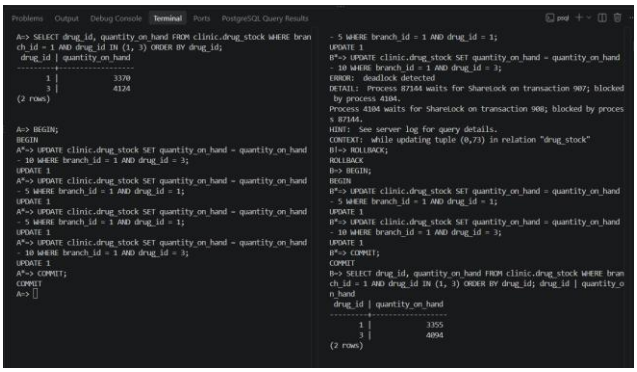


*Figure 2. P3 deadlock detected and resolved by lock ordering*

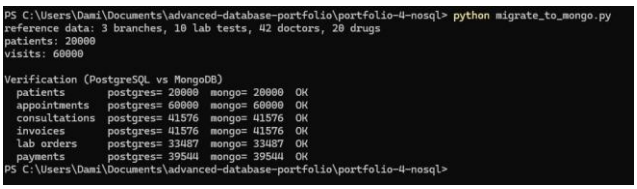



*Figure 3. P4 migration: every PostgreSQL count matched in MongoDB*

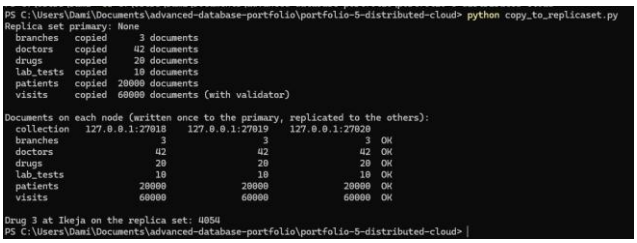



*Figure 4. P5 data written once to the primary, present on all 3 nodes*

## 5. Individual technical reflection

- **Design choices.** Some of my best decisions came from changing my mind. I first wanted a uses_hmo yes/no column on the patient table, then realised it would just repeat what the HMO table already says, so I made it a view instead. I added branch_id to appointments once I remembered that consultants move between branches. In MongoDB I tried to embed only what a doctor reads together on one screen. Sticking with the same clinic all the way through also let me ask both databases the same eight questions and check that the answers matched, which they did.
- **Challenges.** I came into this module new to the terminal, Git, PostgreSQL and MongoDB, so a lot of my time went on things that weren't database problems at all: psql not being recognised, an OpenSSL setting breaking mongosh, running a Python file I hadn't saved, and typing PowerShell commands inside psql. Some mistakes were mine in the demos too. In P3 I forgot a COMMIT, so one test proved nothing until I redid it. In P5 MongoDB cancelled my transaction because I took more than a minute to commit. I kept these in my notes instead of tidying them away, because each one taught me something.
- **What I learned.** The biggest surprise was that an error does not always mean nothing was saved, and "OK" does not always mean the data is safe. I saw both happen in P5. Watching two terminals side by side made isolation levels, deadlocks and the CAP theorem make sense in a way the lecture notes alone didn't. I also got into the habit of measuring rather than guessing: three runs and the median, counting documents on every node, and reading time gaps straight from the data.

## 6. AI use declaration

| | |
|---|---|
| **AI tool used** | Claude (Anthropic), used through the Claude desktop app. |
| **What it was used for** | Planning the portfolio against the brief; step-by-step setup and troubleshooting help (PATH, OpenSSL, psql and mongosh errors); drafting SQL, Python and MongoDB scripts (schema, data generator, queries, demo step guides, migration, replica-set configuration); explaining query plans and error messages; and drafting and rewording the README files, reflection notes and this document from my own results and notes. |
| **How I reviewed, tested and verified the output** | I ran every script and demo myself and checked each result against expected values I worked out by hand: row counts, a running stock trail (4288 → … → 4044), payment IDs and query answers compared between PostgreSQL and MongoDB. Where something was wrong, whether from the AI (a wrong table name, a script printing "primary: None") or my own mistake (a missed COMMIT), I found it, fixed it and recorded it honestly. All screenshots and measured figures are my own outputs. A dated AI-use log is kept in reflection-notes.md in the repository. |
| **Responsibility** | I understand every file submitted and can explain the design decisions, scripts and results if selected for verification. Use of AI does not remove my responsibility for the accuracy and originality of this work. |

## 7. References

Berenson, H. et al. (1995) 'A critique of ANSI SQL isolation levels', Proc. ACM SIGMOD, pp. 1–10. · Brewer, E. (2012) 'CAP twelve years later: how the "rules" have changed', Computer, 45(2), pp. 23–29. · Elmasri, R. and Navathe, S.B. (2016) Fundamentals of Database Systems. 7th edn. Pearson. · MongoDB, Inc. (n.d.) MongoDB Manual (data modeling, schema validation, replication, write concern, transactions, sharding). Available at: https://www.mongodb.com/docs/manual/ (Accessed: 17 October 2026). · PostgreSQL Global Development Group (2025) PostgreSQL 18 Documentation, Ch. 13–14 and 41. Available at: https://www.postgresql.org/docs/18/ · Ports, D.R.K. and Grittner, K. (2012) 'Serializable Snapshot Isolation in PostgreSQL', PVLDB, 5(12), pp. 1850–1861. · Sadalage, P.J. and Fowler, M. (2012) NoSQL Distilled. Addison-Wesley. · Libraries: Faker, psycopg2, pymongo (Python).